




UNIVERSITÉ MARIE ET LOUIS PASTEUR

STGI – Graduate School – Master 2 Internet of Things

3D Catoms Meta-Modules Locomotion in VisibleSim

Author:
Muhammad SHIRAZ

Submitted to:
Dr. Jad BASSIL

October 18, 2026

Abstract

This report presents a distributed algorithm that makes a line of four 3D Catoms meta-modules move to the right in VisibleSim. A meta-module is a ring of 12 catoms. The left-most meta-module is taken apart, its catoms travel one by one through the structure, and they rebuild the same meta-module on the right side, next to the last one. Repeating this operation makes the whole line roll forward. The catoms coordinate only through messages sent to their neighbours, and each catom plans its own path with a breadth-first search. In the demonstration the line performs three complete cycles (12 waves, 144 catom transfers) without any collision. The report explains the algorithm and evaluates its time complexity, its message complexity and its number of motions, using both a theoretical analysis and the numbers measured in the simulator.

Contents

0.1 Problem Statement and Objectives 2
0.2 Assumptions 2
0.3 Algorithm Description 2
0.3.1 Meta-modules, slots and shapes 2
0.3.2 One wave 3
0.3.3 Coordination with messages 3
0.3.4 Movement and path planning 4
0.4 Simulation Results 4
0.5 Complexity Evaluation 7
0.5.1 Number of motions 7
0.5.2 Message complexity 7
0.5.3 Time complexity 7
0.5.4 Memory 8
0.6 Conclusion 8

0.1 Problem Statement and Objectives

The goal of this work is to design a mechanism that lets a set of 3D catoms, arranged as meta-modules, move in a given direction. The starting point is a linear configuration of four meta-modules (green, white, orange and pink). The task is:

- dismantle the left-most meta-module (the green one),
- transport its 12 catoms through the structure,
- and reassemble it on the right side, adjacent to the pink meta-module.

A meta-module can have two symmetrical shapes, called FrontBack (FB) and BackFront (BF). In the line, the shapes alternate: FB, BF, FB, BF. The work to return is the source code, a demonstration video, and this report with the complexity evaluation.

0.2 Assumptions

- The catoms sit on the cells of the FCC lattice of VisibleSim, and every catom knows its own position.
- Communication is only possible between connected neighbours, through the connectors of the catoms.
- A catom can only do the rotations allowed by the VisibleSim motion rules (it rotates around a neighbouring pivot catom).
- The structure stays connected during the whole execution.
- There is no central controller: every catom runs the same block code.

The initial configuration is given in `config.xml`: 48 catoms, 4 meta-modules of 12 catoms, anchors at $x = 4, 8, 12, 16$ with $y = 4$ and $z = 2$. The world is $70 \times 40 \times 60$ cells.

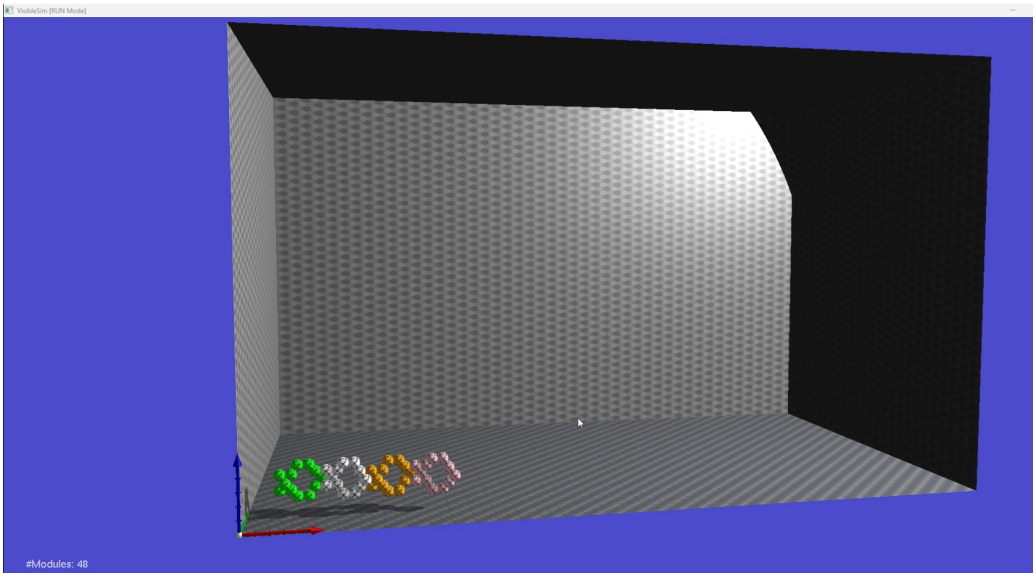


Figure 1: Initial linear configuration: four meta-modules (48 catoms).

0.3 Algorithm Description

0.3.1 Meta-modules, slots and shapes

A *slot* u is a place where one meta-module can stand. Slot u has its anchor (cell $(0, 0, 0)$ of the meta-module) at $x = 4 + 4u$, $y = 4$, $z = 2$. At the beginning, the four meta-modules are in slots

0, 1, 2 and 3. Even slots use the FB shape and odd slots use the BF shape. The 12 cells of each shape are given by the relative positions of the exercise (Table 1); the order of this table is the order used for dismantling and assembling.

Index	FB position	BF position
0	$(-2,-1,3)$	$(-2,0,3)$
1	$(-1,-1,2)$	$(-1,1,2)$
2	$(-2,-1,1)$	$(-2,0,1)$
3	$(-1,-1,3)$	$(-1,0,3)$
4	$(-1,-1,1)$	$(-1,0,1)$
5	$(0,0,4)$	$(0,0,4)$
6	$(0,0,0)$	$(0,0,0)$
7	$(1,0,4)$	$(1,0,4)$
8	$(1,0,0)$	$(1,0,0)$
9	$(1,0,3)$	$(1,-1,3)$
10	$(1,0,1)$	$(1,-1,1)$
11	$(2,1,2)$	$(2,-1,2)$

Table 1: Relative positions of the 12 cells of a meta-module.

At startup, every catom compares its position with the expected cells of each meta-module and finds its two identifiers: the meta-module it belongs to and its index $i \in \{0,\dots,11\}$. It also keeps the slot u where its meta-module currently stands; this value changes after every transfer.

0.3.2 One wave

One *wave* moves the left-most meta-module from slot s to slot $s+4$, just on the right of the last meta-module. Since s and $s+4$ have the same parity, the meta-module is rebuilt with the same shape. The catoms leave in the order of their index $0,1,\dots,11$, and the catom of index i builds the cell of index i of the new meta-module:

$$\text{target}(i) = \text{anchor}(s+4) + \text{RELATIVE}[i].$$

When wave w is finished, wave $w+1$ starts with the next meta-module, which is now the left-most one. One *cycle* is 4 waves: the whole line has moved by its own length (16 cells). The demonstration runs 3 cycles, which is 12 waves.

0.3.3 Coordination with messages

The catom with (meta-module $= 0$, $i = 0$) starts the first wave. After that, the catoms wake each other with a `GO(wave, index, serial)` message. This message is flooded through the structure: each catom that receives it for the first time forwards it to all its other connected neighbours. A catom remembers the messages it already handled (a set of keys $(wave, index, serial)$), so there is no loop. Only the catom whose current slot is the source slot of the wave and whose index matches the message reacts; all the others just forward it.

There are three reasons to send a `GO` message:

1. **Early launch (pipeline).** After a catom has done 1/4 of its route, it wakes the next catom of the order. So four catoms travel at the same time instead of one after the other.
2. **Safety wake.** When a catom arrives at its target, it wakes its successor again. If the successor had no free path before, it tries again.

3. **Next wave.** When the last catom (index 11) of a wave arrives, it wakes catom 0 of the next wave.

A catom that is already moving ignores a new wake-up, so it never starts twice.

0.3.4 Movement and path planning

Each catom finds its path with the breadth-first search (BFS) proposed in the exercise, based on the function `getAllPossibleMotionsFromPosition`. For an empty cell, the function looks at the neighbouring catoms used as pivots and returns the cells that can be reached by a valid rotation. I made one small change: a catom cannot use its own start cell as a pivot, because this cell becomes empty as soon as the catom leaves it.

Because the other catoms also move, a path computed once can become invalid. So the catom plans again from its *current* position before every rotation, and only the first step is executed. Before scheduling it, the code checks `canMoveTo`, so the simulator is never asked for an impossible rotation. The rotation event is scheduled 3 ms after the decision. This small delay lets a message sent just before the rotation leave the catom before it disconnects from its neighbours.

The behaviour of one catom, when it is woken for wave w, is the following:

1. compute the goal cell from the shape of the destination slot;
2. plan with BFS from the current position and execute the first rotation;
3. at the end of each rotation: if the goal is not reached, wake the successor when 1/4 of the route is done (once), then plan the next rotation;
4. when the goal is reached: update the slot and the index, then send the safety wake (or the wake of the next wave if the catom is the last one);
5. if no path is found, stop and wait to be woken again.

0.4 Simulation Results

The simulation was run in VisibleSim with the `config.xml` of the exercise and the real-time scheduler. The window title shows live counters (current wave, transferred catoms, running tasks, messages); these counters are only used for display and the algorithm never reads them. The demonstration video (`3D_Catoms.mp4`) shows the complete execution.

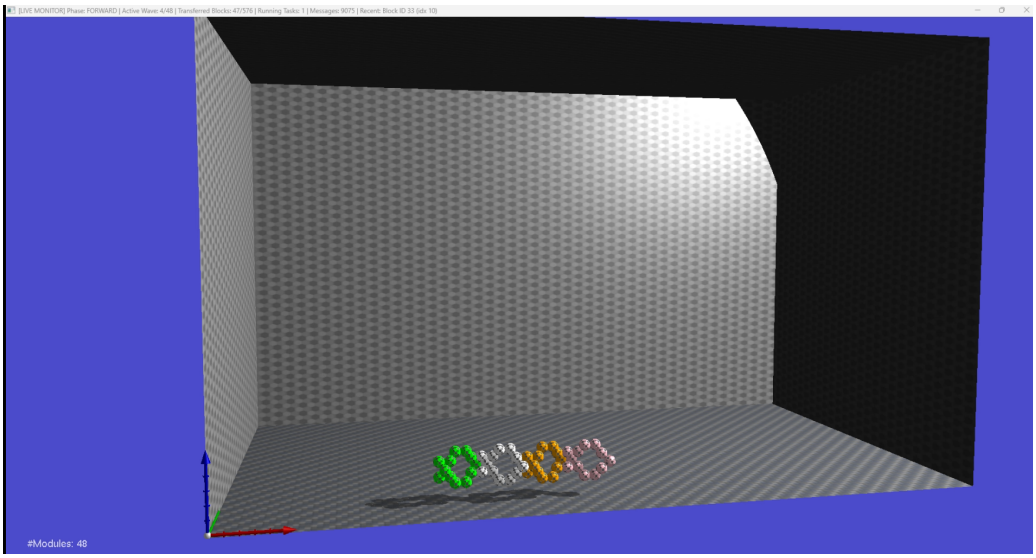


Figure 2: After the first cycle (wave 4 of 48 in the title bar): the line has moved by one meta-module length.

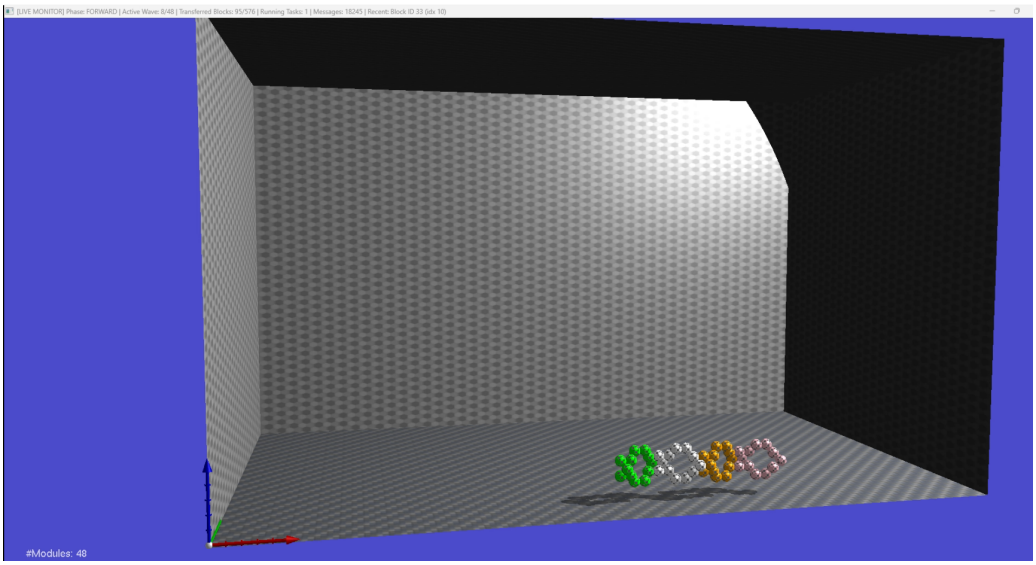


Figure 3: During the second cycle (wave 8): the line keeps its shape while moving to the right.

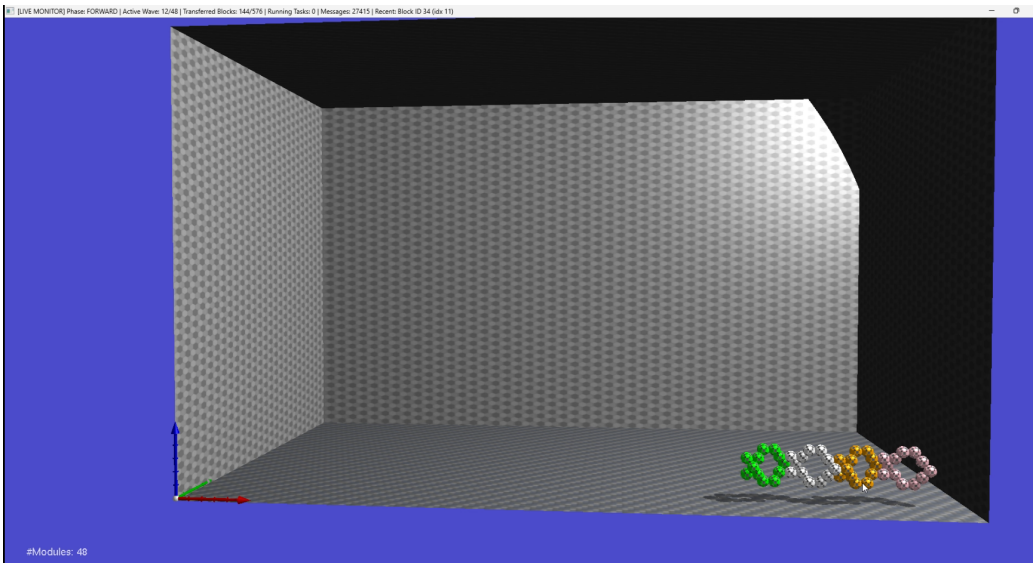


Figure 4: End of the third cycle (wave 12, 144 transferred catoms). The four meta-modules are intact and have moved 48 cells to the right.

Table 2 gives the values printed by the program and the final statistics of VisibleSim.

	Cycle 1	Cycle 2	Cycle 3
Waves done	4	8	12
Catom transfers (total)	48	96	144
Messages (total)	9 075	18 245	27 415

Table 2: Counters read at the end of each cycle.

Number of catoms	48
Waves / catom transfers	12 / 144
Motions (rotations) processed	2 988
Messages processed	27 510
Simulated time	747.4 s
Events processed	305 480

Table 3: Final statistics given by VisibleSim.

Some useful numbers can be deduced from these tables:

- $2\,988/144 \approx 20.8$ rotations for one catom transfer, and about 249 rotations per wave;
- about 2 290 messages per wave, and about 100 messages for each flooded `GO` message;
- about 62 s of simulated time per wave, and about 5.2 s per catom transfer.

During the run, the message "no path yet" appeared several times and "BLOCKED" appeared 6 times (always for the catom of index 2, once every two waves). In all cases the catom stopped, was woken again by the safety wake of its predecessor, and finally reached its target: all 144 transfers were completed and no simulator exception occurred.

0.5 Complexity Evaluation

Let n be the number of meta-modules in the line ($n = 4$), $m = 12$ the number of catoms of a meta-module, and $N = mn$ the total number of catoms. A cycle is n waves, so c cycles give $W = cn$ waves and $M = mW$ catom transfers ($M = 144$ here).

0.5.1 Number of motions

A catom goes from the left end to the right end of the line, a distance of n slots (16 cells here), through the structure. The shortest path found by the BFS has a length $\ell = O(n)$ rotations. In the simulation, the average is 20.8 rotations per transfer, which is about 5.2 rotations per slot crossed. So:

$$\text{motions per wave} = O(m \cdot n), \qquad \text{total motions} = O(M \cdot n) = O(c\,m\,n^2).$$

With m constant, this is $O(c\,n^2)$. Measured: 2 988 motions. For a given meta-module size, the number of motions per catom depends only on the length of the line, not on how many cycles were done.

0.5.2 Message complexity

A flooded message crosses each connection once in each direction, except the one on which it arrived, so it costs at most $2E - N + 1 = O(N)$ messages, where E is the number of connections. In the FCC lattice a catom has at most 12 neighbours, so $E \leq 6N$. Measured: about 100 messages per flood for $N = 48$.

For each wave, the algorithm sends $(m - 1)$ early launches, $(m - 1)$ safety wakes and one next-wave message, which is $2m - 1 = 23$ floods:

$$\text{messages per wave} = (2m-1)\cdot O(N) = O(m^2 n), \qquad \text{total messages} = O(W\,m^2 n) = O(c\,m^2 n^2).$$

With the measured values, $23 \times 100 \approx 2\,300$ per wave, which matches the 2 290 observed (the first wave has 22 floods because nothing wakes it). The total is $O(c\,n^2)$ for constant m. The messages are small (three integers).

The last difference between the final statistic (27 510) and the last counter (27 415) is 95 messages: it is the flood sent when the last catom of wave 12 wakes the first catom of wave 13 (see Section **??**).

0.5.3 Time complexity

Let t_r be the time of one rotation. The time of the messages is small compared to the time of the motions: a flood needs at most $O(n)$ hops (the diameter of the structure), each hop being very short. So the time is dominated by the rotations. The first catom of a wave needs $\ell\,t_r$. With a pipeline of $p = 4$ catoms, a new catom starts every ℓ/p rotations, so one wave takes about

$$T_{wave} \approx \ell\,t_r \left(1 + \frac{m-1}{p}\right) = O\!\left(\frac{m\,n}{p}\right),$$

and the total time is

$$T = W \cdot T_{wave} = O\!\left(\frac{c\,m\,n^2}{p}\right).$$

Measured: 747.4 s for 12 waves, so about 62 s per wave. The pipeline reduces the time of a wave compared with moving one catom at a time ($T_{wave} = O(m\,n)$), but it does not change the order of growth with respect to n.

Measure	Complexity	Measured ($n = 4$, $c = 3$)
Motions	$O(c\,m\,n^2)$	2 988
Messages	$O(c\,m^2 n^2)$	27 510
Time	$O(c\,m\,n^2/p)$	747.4 s

Table 4: Summary of the complexity evaluation.

0.5.4 Memory

Each catom stores a few integers (its meta-module, index, slot, goal and counters) and the set of messages already handled. This set grows by one key for each flood received, so it is limited by $O(W\,m)$ entries per catom.

0.6 Conclusion

This work implements a fully distributed locomotion of a line of 3D Catoms meta-modules. The left-most meta-module is dismantled, its catoms move one after another (four at the same time) through the structure and rebuild the meta-module on the right side. In the simulation, the line moved by three cycles without losing its shape: 12 waves, 144 transfers, 2 988 rotations and 27 510 messages in 747.4 s of simulated time. The analysis shows that the number of motions, the number of messages and the time all grow quadratically with the length of the line for a given number of cycles. The most important improvement for the future is to reduce the number of messages and to complete the up-left extension.